



Soluciones IoT - EDGE Computing

Agustin Candia - Diego Graselli - Matías Pagano - Diego Vilches



UNIVERSIDAD
NACIONAL
DE LA PLATA

REPASO

Edge IoT + Machine Learning

¿Qué es Edge IoT?

Procesamiento local de datos cerca del sensor.

Objetivos

- menor consumo de red
- menor latencia
- optimización energética
- operación offline y respuesta en tiempo real
- privacidad de datos

REPASO

¿Qué es Machine Learning?

Técnicas que permiten:

- aprender patrones a partir de datos,
- clasificar estados,
- detectar anomalías,
- realizar inferencias.

¿Qué es TinyML?

Machine Learning ejecutándose en microcontroladores. Ejemplos:

- ESP32
- Arduino
- STM32

Introducción - Mantenimiento predictivo

¿Qué es?

El mantenimiento predictivo busca:

- detectar comportamientos anómalos,
- anticipar fallas,
- evitar roturas,
- reducir costos operativos.

Idea principal

Analizar vibraciones para identificar:

- desbalanceos,
- desgaste,
- rodamientos defectuosos,
- resonancias,
- cambios de comportamiento.

Introducción - Análisis de caso en clase

TinyML aplicado a vibraciones

Objetivo

Clasificar estados de un ventilador utilizando:

- ESP32
- MPU6050
- Edge Impulse

Clases a detectar

- Ventilador apagado
- Potencia baja
- Potencia alta

Introducción - Acelerómetro

¿Qué mide?

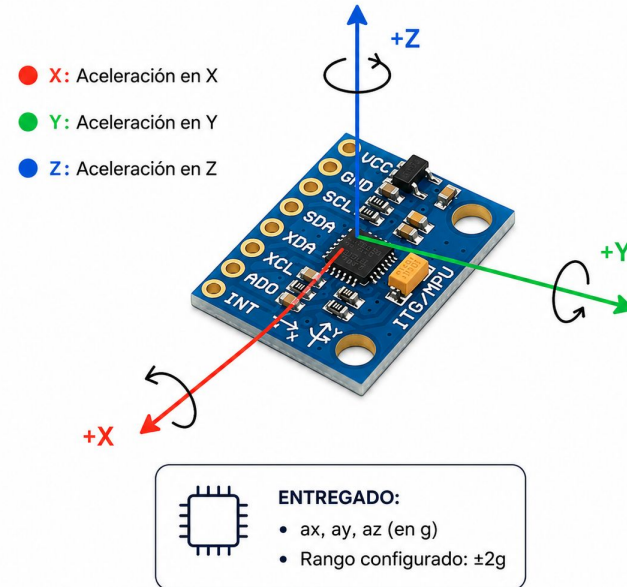
- Mide aceleración en eje X,Y y Z, CUando está quieto la única aceleración presente es la gravedad (9.81m/s^2) dirigida hacia abajo

Sensor MEMS

- masa microscópica suspendida
- medición capacitiva
- detecta vibraciones y gravedad

¿Por qué $\pm 2\text{g}$?

- mayor sensibilidad
- mejor resolución para vibraciones pequeñas



Introducción - Acelerómetro

De 3 ejes a 1 sola señal. La gravedad puede repartirse entre X/Y/Z, pero la magnitud total sigue siendo similar.

```
float mag = sqrt(ax*ax + ay*ay + az*az);
```

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Ventajas

- simplifica el modelo
- reduce dimensionalidad
- mejora robustez frente a orientación

Entonces:

- el sensor puede rotarse
- horizontal / vertical / inclinado
- el sistema sigue funcionando razonablemente bien

IMPORTANTE: La orientación importa poco. El **MONTAJE** mecánico importa **MUCHO**. (Rigidez de la fijación, cercanía al motor, vibraciones estructurales, resonancias, contacto mecánico, etc)

Introducción - Acelerómetro

Frecuencia de muestreo e importancia

- 50 muestras por segundo
- una muestra cada 20 ms

$$T = \frac{1000}{50} = 20ms$$

```
const float SAMPLE_RATE_HZ = 50.0f;
```

La **frecuencia de muestreo** define qué tan rápido “observamos” la vibración, determinando cuánta información vibratoria podemos capturar del sistema mecánico. Con una **Frecuencia baja** de muestreo : pérdida de detalle, aliasing, frecuencias falsas, mala detección de vibraciones rápidas. Con una **Frecuencia alta** de muestreo tendremos mejor resolución, más detalle espectral, mejor análisis de vibraciones **PERO** mayor uso de CPU, más RAM, datasets más grandes. modelos ML más pesados

FRECUENCIA DE MUESTREO E IMPORTANCIA

Ejemplo: Ventilador de 1200 RPM

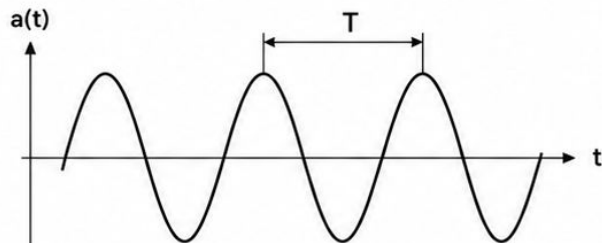
1. SISTEMA REAL

Ventilador



2. SEÑAL GENERADA

Vibración periódica



1200 RPM = 20 Hz

3. FRECUENCIA DE MUESTREO

$F_s = 50$ Hz

50 muestras por segundo

$T_s = 20$ ms

Muestras por ciclo:

$$\frac{F_s}{f} = \frac{50}{20} = 2.5$$

~ 2.5 muestras por vuelta

Alcanza para detectar la frecuencia principal, pero con **poca resolución**.

4. MUESTREO DE LA SEÑAL



IDEA CLAVE

La frecuencia de muestreo define cuánta información vibratoria podemos capturar del sistema mecánico.

5. REGLA DE NYQUIST

Para detectar correctamente una frecuencia:

$$f_{\max} = \frac{f_s}{2}$$

Con $f_s = 50$ Hz -> $f_{\max} = 25$ Hz

Podemos analizar correctamente hasta 25 Hz.

6. CONSECUENCIAS PRÁCTICAS



Frecuencia baja de muestreo

- Pérdida de detalle
- Aliasing (frecuencias falsas)
- Mala detección de vibraciones rápidas
- Menor precisión del modelo ML



Frecuencia alta de muestreo

- Mejor resolución
- Más detalle espectral
- Mejor análisis de vibraciones
- Mejor desempeño del modelo ML

Pero implica:

- Mayor uso de CPU
- Más RAM
- Datasets más grandes
- Modelos ML más pesados

Introducción - Edge Impulse



Ventajas pedagógicas

- simplifica TinyML
- reduce barrera de entrada
- pipeline visual completo
- deployment rápido en ESP32

Ventajas técnicas

- generación automática de features DSP
- entrenamiento simplificado
- exportación directa a ESP32
- optimización para microcontroladores
- inferencia en tiempo real

Permite enfocarse en conceptos

- sensores
- adquisición de datos
- DSP / FFT
- Machine Learning
- inferencia en Edge
- clasificación de vibraciones

sin quedar bloqueado por:

- frameworks complejos,
- infraestructura ML,
- tooling avanzado.

Transición a Herramientas más avanzadas



EDGE IMPULSE

Plataforma completa para TinyML



Adquisición de datos

Desde el dispositivo



DSP / Feature Engineering

FFT y transformaciones automáticas



Entrenamiento de modelos

Interfaz guiada y simple



Deployment

Exportación directa a
ESP32, Arduino, etc.



Inferencia en el dispositivo

En tiempo real, optimizado



¿Por qué hacer la transición?

- ✓ Necesitamos más control sobre el modelo y los datos
- ✓ Modelos más complejos o personalizados
- ✓ Integración con otros sistemas y herramientas
- ✓ Investigación y aplicaciones más avanzadas



TensorFlow



Keras

Deep Learning profesional

- Keras: API de alto nivel para construir modelos fácilmente
- TensorFlow: motor potente y flexible
- TensorFlow Lite / TFLM: despliegue en dispositivos



PyTorch

- Framework dinámico y flexible
- Ideal para investigación
- Gran ecosistema y comunidad



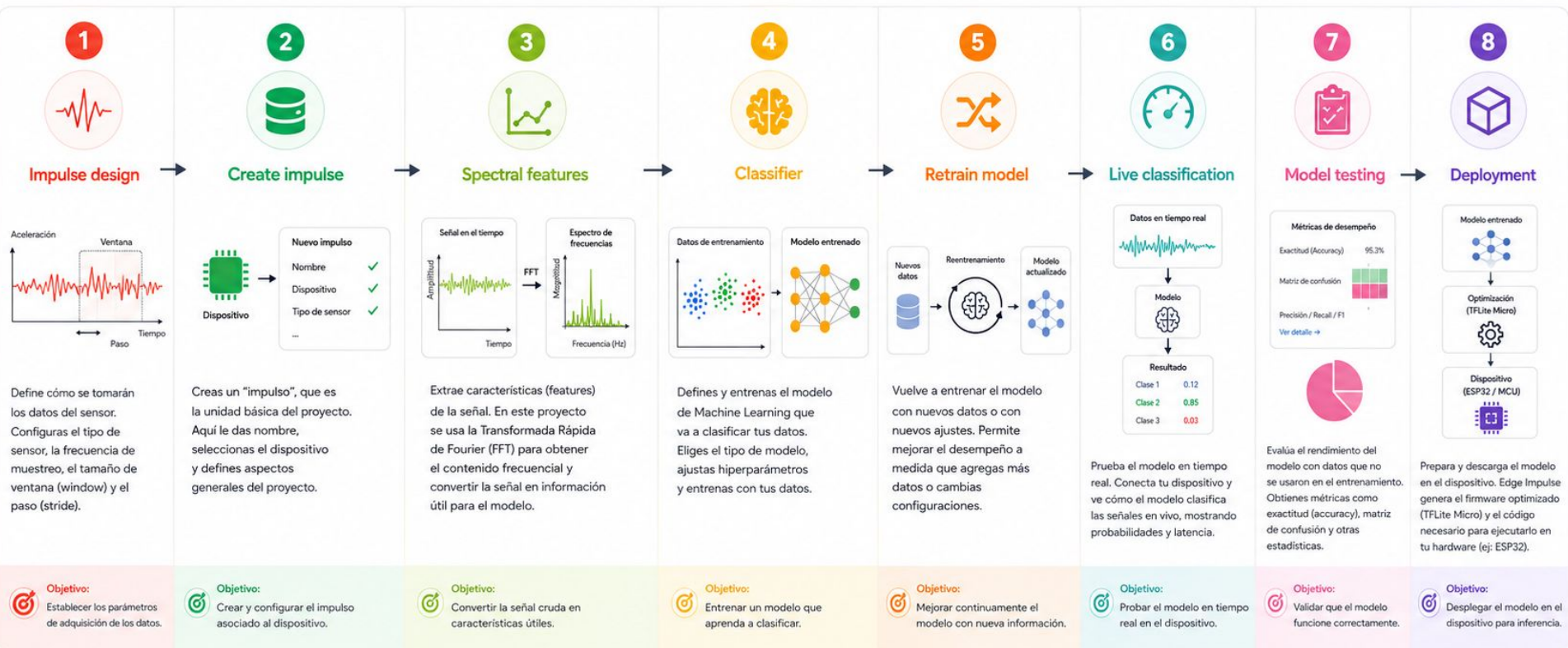
Scikit-learn

- Machine Learning clásico
- Algoritmos tradicionales
- Ideal para tabular data y prototipos rápidos

Pipeline de Edge Impulse



De los datos del sensor a un modelo de TinyML desplegado en el dispositivo



Resumen del flujo

Diseñas la adquisición de datos



Creas el impulso y configuras el proyecto



Extraes características de la señal



Entrenas el modelo para clasificar



Mejoras el modelo iterativamente



Pruebas en vivo en el dispositivo



Evalúas el rendimiento del modelo



Desplegas el modelo en el dispositivo

Este pipeline te permite ir desde la captura de datos con sensores hasta la inferencia en el borde (Edge) de forma rápida y guiada.

Funciona nuestro modelo ? Metricas....

Accuracy (Exactitud)

¿Qué mide?

Porcentaje total de predicciones correctas.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Ejemplo

Si el modelo evalúa 100 muestras y acierta 92 ==> 92%

Importante: Un accuracy alto **NO** garantiza que todas las clases funcionen bien, ni que el modelo sea robusto (Especialmente cuando el dataset está desbalanceado, hay clases minoritarias, algunas clases son más difíciles.).

Funciona nuestro modelo ? Métricas....

Confusion Matrix

La matriz de confusión permite entender no sólo cuánto acierta el modelo, sino específicamente dónde se equivoca y qué clases generan más confusión

Interpretación

- diagonal principal: aciertos.
- fuera de diagonal: errores y confusiones.

Muy útil para detectar:

- clases problemáticas,
- datasets desbalanceados,
- errores sistemáticos.

Real \ Predicho	Apagado	Potencia baja	Potencia alta
Apagado	28	2	0
Potencia baja	3	24	3
Potencia alta	0	4	26

$$\text{Accuracy: } \frac{28 + 24 + 26}{90} = 86.7\%$$

Funciona nuestro modelo ? Métricas....

Precision

Precision mide qué tan confiables son las predicciones del modelo. Responde a la pregunta: “Cuando el modelo dice una clase, ¿realmente era esa clase?”

Ejemplo:

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

Supongamos que el modelo detecta “Potencia Alta” 10 veces.

Pero al verificar:

- 8 casos eran realmente “Potencia Alta”,
- 2 casos eran incorrectos.

$$Precision = \frac{8}{10} = 80\%$$

Importante : Una precisión baja produce falsas alarmas, clasificaciones incorrectas, pérdida de confianza en el sistema.

Funciona nuestro modelo ? Métricas....

Recall

Recall mide la capacidad de detección del modelo. Responde a la pregunta: “De todos los casos reales, ¿cuántos logró detectar?”

Ejemplo

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Supongamos que realmente existían 10 casos de “Potencia Alta”

Pero el modelo detectó correctamente sólo:

- 7 casos.

$$\text{Recall} = \frac{7}{10} = 70\%$$

(El modelo dejó escapar 3 casos reales.)

Importante : Un recall bajo implica fallas no detectadas, eventos importantes perdidos, menor capacidad de detección del sistema. En mantenimiento predictivo esto es crítico, porque una falla no detectada puede terminar en una rotura real del equipo.

Funciona nuestro modelo ? Métricas....

F1 Score

El F1 Score es una métrica que combina Precisión y Recall en un único valor balanceado permitiendo evaluar simultáneamente la confiabilidad y la capacidad de detección del modelo.

Se utiliza ya que muchas veces:

- mejorar Precision empeora Recall,
- y mejorar Recall empeora Precision.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

El **F1** busca un equilibrio entre ambas. Es importante porque un modelo puede tener **Precision** alta pero perder muchos casos reales. **Recall** alto pero generar muchas falsas alarmas.



UNIVERSIDAD
NACIONAL
DE LA PLATA